

VoiceRecom 프로젝트 계획서

1. 개요

1.1 목적

본 프로젝트는 노래 연습자가 본인의 음성을 업로드하면 AI가 음정과 박자를 기반으로 분석해 객관적인 피드백을 제공, 종합 점수 92점 이상 달성 시 해당 곡의 완창으로 인정하는 한 곡 완창 챌린지 서비스를 개발하는 것을 목적으로 한다.

단순한 점수 제공에 그치지 않고, 어느 구간에서 음이 이탈하고 박자가 어긋났는지, 그 원인은 무엇인지까지 분석해 사용자가 실질적으로 개선할 수 있는 방향을 제시한다. 또한 반복 도전을 통한 점수 추이의 시각화로 사용자 스스로 성장과정을 확인, 지속적인 참여의 동기를 부여해 개인화된 보컬 트레이닝 경험을 제공하는 것을 목표로 한다.

1.2 필요성

- 객관적 자기 평가 수단의 부재
: 자신의 음정과 박자의 정확성을 판단하는 것에 대한 어려움
- 전문 피드백의 높은 접근 장벽
: 전문 보컬 코치에게 피드백을 받으려면 상당한 비용과 시간을 필요로 하기에, 노래를 취미로 즐기는 일반인이 전문적인 피드백을 받을 기회는 매우 제한적이다.
- 지속적 연습을 유도하는 구조의 필요성
: 많은 사용자가 노래 연습을 시작해도 뚜렷한 목표나 성취감 없이 중도에 포기하는 경우가 많다. 완창이라는 명확한 달성 기준과 점수 추이를 통해 구체적인 목표와 자신의 성장을 확인하고 꾸준한 연습을 유도한다.

1.3 기대효과

- 학습 효율 향상
- 보컬 피드백의 접근성 및 편의성 향상
- 신개념 서비스 모델 시도

1.4 활용

- 개인 보컬 연습 도구/노래방, 음악 플랫폼 연계/챌린지 문화 콘텐츠화

1.5 계획서의 변경기록

26.03.20 초안 작성

1.6 참고문헌

Google OAuth 2.0 API 문서, Librosa 라이브러리 레퍼런스, PESTO 라이브러리 레퍼런스, FastAPI 문서, AudioAPI 문서

1.8 정의와 약어

DTW : Dynamic Time Warping 알고리즘의 약자로,

센트 : 음정 차이를 느끼는 단위로, 1옥타브는 1200개의 센트로 나뉘며, 반음은 100센트이다.

2. 프로젝트 조직

2.1 프로세스 모델

소규모 애자일 모델

2.2 조직 구조

연제환 : 팀장, 백엔드 개발, 기획·PM, 전체 방향 설정

손하은 : 프론트엔드 개발, UI/UX 구현 담당

임혜성 : AWS배포, 운영환경 구성, 데이터 처리 시스템 구현, QA 담당

2.3 조직의 범위와 인터페이스

본 프로젝트는 소규모 독립 팀으로 운영되어 해당 사항 없음.

2.4 프로젝트 책임

각 주차별로 마일스톤을 파트별로 관리.

3. 관리적 프로세스

3.1. 관리적 목적과 우선순위

1순위 - 음성 분석 과정의 구현과 정확도 확보

2순위 - 사용자 경험 최적화(응답시간 최소화 등)

3순위 - 게이미피케이션, 로딩 팁 등 추가적 요소 구현.

3.2 가정과 제한

소셜 로그인을 사용자 간 인증 수단으로 사용하며, 웹 브라우저 환경에서의 마이크 접근 권한 또는 이미 기존에 녹음된 녹음 파일이 필요함.

3.3 위험관리

음성 데이터 처리 시간 지연 문제, AI 피드백의 정확성, 신뢰성 확보 문제, 서버 트래픽 관리

3.4 통제 메커니즘

주간 비정기 회의, 노션을 이용한 마일스톤 설정 및 코드, mvp 리뷰.

3.5 인력(2.3에 기술해놓았기에 생략)

4. 기술적 프로세스

4.1 방법론 도구

React/Next.js(프론트엔드)

FastAPI-Python(백엔드)

Figma(디자인)

SQLite(DB)

Librosa, PESTO(AI/ML)

4.2 소프트웨어 문서화

API 명세서(Swagger), DB 설계도, UI/UX 와이어프레임

4.3 지원기능

4.3.1 회원관리

- 회원가입 : 사용자는 Google 로그인을 통하여 가입할 수 있다.

- 로그인 : 사용자는 Google 로그인을 통하여 로그인할 수 있다.

- 로그아웃 : 회원가입 및 로그인 상태인 사용자에게 한해 로그아웃 기능을 지원한다.

4.3.2 피드백 받을 노래 선정

- 노래 추천

사용자는 "노래 추천 받기" 버튼을 클릭하여 노래 추천 기능을 이용할 수 있다.

버튼 클릭 시 목소리 분석용 음성파일을 업로드하고, 시스템은 음성을 분석하여 사용자의 목소리와 어울리는 노래 5곡을 리스트 형태로 제공한다.

- 사용자 선택

사용자는 "직접 선택하기" 버튼을 클릭하여 노래를 직접 선정할 수 있다.

버튼 클릭 시 DB에 등록된 노래 리스트 기반으로 검색하여 원하는 노래를 선택할 수 있다.

4.3.3 음성파일 업로드

- 사용자는 본인이 노래를 부른 음성 파일을 업로드할 수 있다.

- 업로드한 파일을 삭제하고 다른 파일을 업로드할 수 있다.

- "분석하기" 버튼을 클릭하여 업로드한 파일을 바탕으로 AI 분석을 요청할 수 있다.

4.3.4 AI기반 분석

- 업로드된 음성 파일은 원곡과 비교하여 음정 및 박자 두 가지 영역을 분석한다.

- 사용자의 음색(톤)과 무관하게 음 높낮이만을 추출하여 원곡과 비교한다.

- 음정 점수와 박자 점수를 각각 독립적으로 산출, 가중치를 적용해 종합 점수를 산출한다.

4.3.5 이전 도전과의 비교 피드백

- 동일한 노래에 대해 2회 이상 도전한 경우, 직전 도전과의 점수 변화를 비교하여 제공한다.

- 이전 도전 대비 개선된 영역과 아직 부족한 영역을 구분하여 텍스트로 안내한다.

4.3.6 한 곡 완창 완료

- 종합 점수가 92점 이상인 경우 해당 노래를 마스터한 것으로 간주한다.

4.3.7 마이페이지

- 받은 피드백 모아보기

사용자는 각 노래에 대한 피드백을 최신순으로, 노래 목록으로 제공한다.

완창한 곡/도전 중인 곡으로 분리되어 있으며, 목록에서 특정 곡을 클릭하면 해당 곡에 대한 상세 피드백 페이지로 이동한다.

- 노래별 상세 피드백

해당 노래에 대한 모든 도전 기록을 확인할 수 있으며,

그래프/피드백 두 가지 탭으로 구성된다.

그래프는 회차별 음정/박자/종합 점수를 그래프로 시각화하여 제공하며,

피드백 탭은 회차별 AI 텍스트 피드백을 최신순으로 목록 형태로 제공한다.

4.3.8 동기부여 게이미피케이션

- 뱃지&칭호 시스템

첫 완창, 3곡 완창, 장르별 완창 등 다양한 달성 조건에 따른 뱃지를 부여한다.

- 챌린지 히스토리

도전 횟수, 총 연습 시간, 완창한 곡, 최고점 등 통계를 제공해 성장을 확인할 수 있다.

- 연속 도전 스트릭

매일 연습하면 스트릭(연속 일수)를 표시해서 서비스 지속 이용을 유도한다.

4.3.9 비기능적 요구사항

- 동시 접속자 급증에 대한 서비스 지연 및 처리 문제가 없어야 한다.

- 분석은 5초 이내에 완료되어 결과를 사용자에게 표출해야 한다.

- AI 분석의 신뢰성이 90% 이상을 유지해야 한다.

- 사용자의 개인정보는 분석 시 필요한 음성 데이터뿐이며, 피드백 생성 후 즉시 파기한다.

(더 상세한 내용, 수치 기준은 별도 첨부한 요구사항 명세서를 참고)

5. 작업, 스케줄, 예산

5.1 작업 스케줄

	프론트엔드	백엔드	AI
기획 심화 (4주차)			
4주차	주제 구체화 및 기획 검토 — 요구사항 재정의, 기술 스택 확정, 역할 분담		
기반 구축 (5~6주차)			
5주차	환경 세팅 Google 로그인 UI	DB 설계 Google OAuth 구현	PESTO / DTW 라이브러리 검토 및 환경 세팅
6주차	노래 선정 UI (추천 / 직접 선택)	노래 DB 구축 노래 검색 API	목소리 분석 모델 설계 노래 추천 로직 구현
핵심 기능 개발 (7~9주차)			
7주차	음성 파일 업로드 UI 분석 요청 버튼	파일 업로드 API 유효성 검사 (형식/용량)	음성 분석 파이프라인 구현 (PESTO)
8주차	피드백 결과 화면 UI (점수 + 텍스트)	분석 요청 처리 API 피드백 저장 로직	박자 분석 파이프라인 구현 (DTW) 점수 산출 로직
9주차	이전 도전 비교 피드백 UI	도전 이력 관리 API 비교 피드백 API	텍스트 피드백 생성 로직 (결과 분석 + 원인 분석)
마이페이지 & 완성도 (10~12주차)			
10주차	마이페이지 UI (노래 목록, 완창/도전 분리)	마이페이지 데이터 API 완창 판정 로직 (92점)	모델 정확도 검증 및 튜닝
11주차	노래별 상세 피드백 페이지 (그래프 탭 / 피드백 탭)	점수 추이 데이터 API	이전 도전 비교 분석 로직 완성
12주차	프론트 ↔ 백 ↔ AI 전체 연동 및 통합 테스트		
마무리 & 발표 (13~15주차)			
13주차	UI/UX 개선 예외 처리 보완	성능 최적화 보안 점검	엣지 케이스 대응 피드백 품질 개선
14주차	최종 버그 수정 및 데모 시연 준비 / 문서 정리		
15주차	최종 발표		